

Лабораторная работа №4

Моделирование конфликта Защитник-Нападающий

Аристова Арина Олеговна

Содержание

1	Цель работы	4
2	Задание	5
3	Выполнение лабораторной работы	6
3.1	Создание проекта	6
3.2	Основной модуль	7
3.3	Запуск симуляций	12
3.4	Построение графиков	13
3.5	Контрольные вопросы	19
4	Выводы	22
	Список литературы	23

Список иллюстраций

3.1	Инициализация проекта	6
3.2	Установка пакетов с помощью скрипта	6
3.3	Скрипт основного модуля	7
3.4	Скрипт запуска симуляций	12
3.5	Скрипт построения графиков	14
3.6	Запуск скрипта	17
3.7	Зависимость вероятности атаки на первый актив p_1 от отношения ценностей V_1/V_2	17
3.8	Средний выигрыш Нападающего	18
3.9	Запуск генерации производных форматов	18
3.10	Полученные файлы других форматов	19

1 Цель работы

Освоить применение теории игр для анализа противостояния в сфере информационной безопасности. Научиться строить матричную игру, находить равновесие Нэша в чистых и смешанных стратегиях.

2 Задание

- Формализовать конфликт «Защитник–Нападающий» в виде антагонистической игры.
- Реализовать на Julia функции расчёта платёжной матрицы, поиска равновесия Нэша и симуляции игры.
- Визуализировать зависимость ожидаемого выигрыша и равновесных вероятностей от стоимости защиты и величины ущерба.

3 Выполнение лабораторной работы

3.1 Создание проекта

Я инициализировала проект и установила необходимые пакеты для дальнейшей работы с помощью скрипта *add_packages.jl*

```
julia> using DrWatson
julia>

julia> initialize_project("project"; authors="Ваше Имя", git=false)
Activating new project at `C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab04\project`
Resolving package versions...
Installed ScopedValues - v1.6.2
Installed JLLWrappers — v1.8.0
Installed JLD2 — v0.6.4
Installed FileIO — v1.19.0
Updating `C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab04\project\Project.toml`
[634d3b9d] + DrWatson v2.19.1
Updating `C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab04\project\Manifest.toml`
[0b6fb165] + ChunkCodecCore v1.0.1
```

Рисунок 3.1: Инициализация проекта

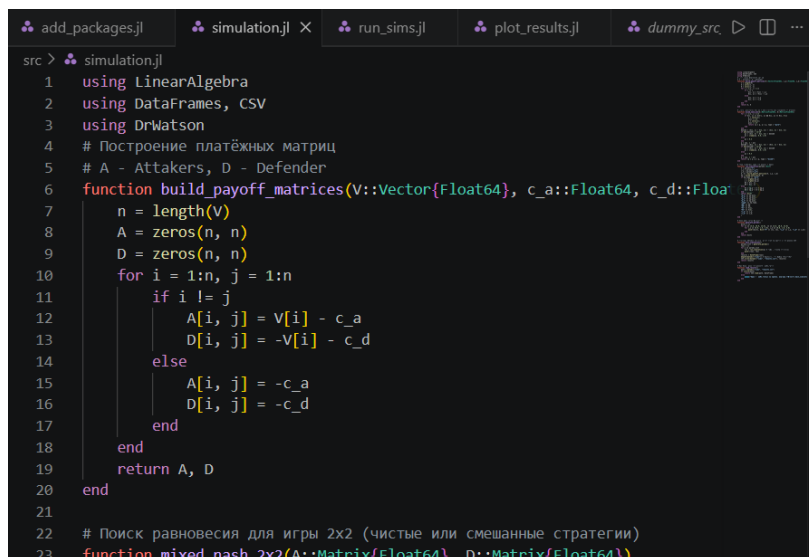
```
add_packages.jl × simulation.jl run_sims.jl plot_results.jl dummy_src ▸ □ ...
add_packages.jl
1 using Pkg
2 Pkg.activate(".") # Активируем текущий проект
3 ## ОСНОВНЫЕ ПАКЕТЫ ДЛЯ РАБОТЫ
4 packages = [
5     "DrWatson", # Организация проекта
6     "DifferentialEquations", # Решение ОДУ
7     "Plots", # Визуализация
8     "DataFrames", # Таблицы данных
9     "CSV", # Работа с CSV
10    "JLD2", # Сохранение данных
11    "Literate", # Literate programming
12    "IJulia", # Jupyter notebook
```

Рисунок 3.2: Установка пакетов с помощью скрипта

3.2 Основной модуль

Файл *src/simulation.jl* является основным модулем, используемым для моделирования. Он содержит несколько функций:

- ***build_payoff_matrices***(*V::Vector{Float64}*, *c_a::Float64*, *c_d::Float64*) строит две платёжные матрицы игры размера $n \times n$
- ***mixed_nash_2x2***(*A::Matrix{Float64}*, *D::Matrix{Float64}*) ищет равновесие Нэша для биматричной игры 2×2
- ***run_simulation***(*params::Dict*) проводит один эксперимент с заданными параметрами игры
- ***generate_params***() формирует полную сетку параметров для симуляционного эксперимента. Перебирает все комбинации:
 - ценностей $V1 \in \{5, 10, 15\}$, $V2 \in \{5, 10, 15\}$
 - затрат $c_a \in \{0, 1, 3\}$, $c_d \in \{0, 1, 3\}$
- ***main_simulations***(): Основная функция запуска всех симуляций.
- ***load_results***() загружает ранее сохранённые результаты из CSV-файла



```
src > simulation.jl
1 using LinearAlgebra
2 using DataFrames, CSV
3 using DrWatson
4 # Построение платёжных матриц
5 # A - Attakers, D - Defender
6 function build_payoff_matrices(V::Vector{Float64}, c_a::Float64, c_d::Float64)
7     n = length(V)
8     A = zeros(n, n)
9     D = zeros(n, n)
10    for i = 1:n, j = 1:n
11        if i != j
12            A[i, j] = V[i] - c_a
13            D[i, j] = -V[i] - c_d
14        else
15            A[i, j] = -c_a
16            D[i, j] = -c_d
17        end
18    end
19    return A, D
20 end
21
22 # Поиск равновесия для игры 2x2 (чистые или смешанные стратегии)
23 function mixed_nash_2x2(A::Matrix{Float64}, D::Matrix{Float64})
```

Рисунок 3.3: Скрипт основного модуля

```

cd(joinpath(@__DIR__, "../project"))
using LinearAlgebra
using DataFrames, CSV
using DrWatson

```

Построение платёжных матриц A - Attakers, D - Defender

```

function build_payoff_matrices(V::Vector{Float64}, c_a::Float64, c_d::Float64)
    n = length(V)
    A = zeros(n, n)
    D = zeros(n, n)
    for i = 1:n, j = 1:n
        if i != j
            A[i, j] = V[i] - c_a
            D[i, j] = -V[i] - c_d
        else
            A[i, j] = -c_a
            D[i, j] = -c_d
        end
    end
    return A, D
end

```

build_payoff_matrices (generic function with 1 method)

Поиск равновесия для игры 2x2 (чистые или смешанные стратегии)

```

function mixed_nash_2x2(A::Matrix{Float64}, D::Matrix{Float64})
    for i = 1:2, j = 1:2
        if A[i, j] >= A[3-i, j] && D[i, j] >= D[i, 3-j]

```



```

        p = zeros(2);
        p[i] = 1.0
        q = zeros(2);
        q[j] = 1.0
        return (p = p, q = q, type = "pure")
    end
end
denomA = (A[1, 1] - A[2, 1]) - (A[1, 2] - A[2, 2])
if abs(denomA) > 1e-10
    q1 = (A[2, 2] - A[1, 2]) / denomA
    q1 = clamp(q1, 0.0, 1.0)
else
    q1 = 0.5
end
q = [q1, 1 - q1]
denomD = (D[1, 1] - D[1, 2]) - (D[2, 1] - D[2, 2])
if abs(denomD) > 1e-10
    p1 = (D[2, 2] - D[2, 1]) / denomD
    p1 = clamp(p1, 0.0, 1.0)
else
    p1 = 0.5
end
p = [p1, 1 - p1]
return (p = p, q = q, type = "mixed")
end

```

mixed_nash_2x2 (generic function with 1 method)

Одна симуляция (без сохранения в файл)

```

function run_simulation(params::Dict)

    V = params["V"]
    c_a = params["c_a"]
    c_d = params["c_d"]
    A, D = build_payoff_matrices(V, c_a, c_d)
    eq = mixed_nash_2x2(A, D)

    if eq.type == "pure"
        i = argmax(eq.p)
        j = argmax(eq.q)
        UA = A[i, j]
        UD = D[i, j]
    else
        UA = eq.p' * A * eq.q
        UD = eq.p' * D * eq.q
    end

    return Dict(
        "p_1" => eq.p[1],
        "p_2" => eq.p[2],
        "q_1" => eq.q[1],
        "q_2" => eq.q[2],
        "type" => eq.type,
        "UA" => UA,
        "UD" => UD,
        "V1" => V[1],
        "V2" => V[2],
        "c_a" => c_a,
        "c_d" => c_d,
    )

```

end

run_simulation (generic function with 1 method)

Генерация сетки параметров

```
function generate_params()
    dicts = []
    for v1 in [5.0, 10.0, 15.0], v2 in [5.0, 10.0, 15.0]
        for c_a in [0.0, 1.0, 3.0], c_d in [0.0, 1.0, 3.0]
            push!(dicts, Dict{"V" => [v1, v2], "c_a" => c_a, "c_d" => c_d})
        end
    end
    return dicts
end
```

generate_params (generic function with 1 method)

Основная функция: прямой расчёт всех вариантов и сохранение CSV

```
function main_simulations()
    params_list = generate_params()
    rows = []
    for p in params_list
        res = run_simulation(p) # теперь всегда вычисляем
        push!(rows, res)
    end
    results = DataFrame(rows)
    mkpath(datadir("sims")) # убедимся, что папка существует
    CSV.write(datadir("sims", "results.csv"), results)
    return results
end
```

main_simulations (generic function with 1 method)

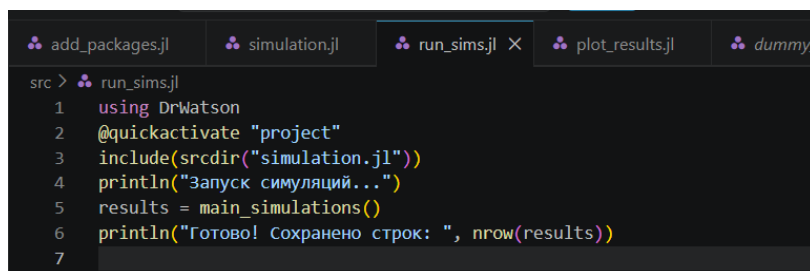
Загрузка ранее сохранённых результатов

```
function load_results()
    path = datadir("sims", "results.csv")
    if isfile(path)
        return CSV.read(path, DataFrame)
    else
        error("Файл с результатами не найден. Сначала выполните main_simulations().")
    end
end
```

load_results (generic function with 1 method)

3.3 Запуск симуляций

Что делает: - Печатает сообщение о запуске. - Выполняет main_simulations() (все комбинации параметров). - Выводит количество рассчитанных вариантов.



```
src > run_sims.jl
1 using DrWatson
2 @quickactivate "project"
3 include(scrdir("simulation.jl"))
4 println("Запуск симуляций...")
5 results = main_simulations()
6 println("Готово! Сохранено строк: ", nrow(results))
7
```

Рисунок 3.4: Скрипт запуска симуляций

```
cd(joinpath(@__DIR__, "../project"))
using DrWatson
@quickactivate "project"
```

```
include(scrdir("simulation.jl"))
println("Запуск симуляций...")
results = main_simulations()
println("Готово! Сохранено строк: ", nrow(results))
```

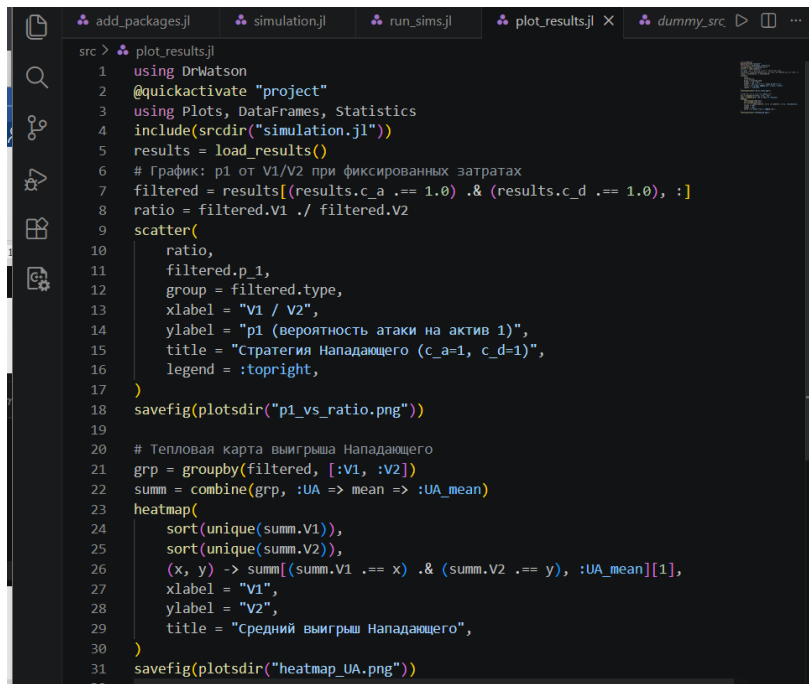
Запуск симуляций...

Готово! Сохранено строк: 81

3.4 Построение графиков

Файл *scripts/plot_results.jl*:

- Загружает результаты через `load_results()`.
- Фильтрует строки с `c_a = 1.0` и `c_d = 1.0`.
- Строит точечный график (scatter). Зависимость вероятности атаки на первый актив `p1` от отношения ценностей `V1/V2` (рис. 4.1). Точки разделены по типу равновесия (pure/mixed). Сохраняет в `plots/p1_vs_ratio.png`.
- Группирует отфильтрованные данные по `V1` и `V2`, вычисляет средний ожидаемый выигрыш Нападающего `UA_mean`.
- Строит тепловую карту (heatmap). Средний выигрыш Нападающего в координатах (`V1`, `V2`) (рис. 4.2). Сохраняет в `plots/heatmap-UA.png`.



```
src > plot_results.jl
1 using DrWatson
2 @quickactivate "project"
3 using Plots, DataFrames, Statistics
4 include(scrdir("simulation.jl"))
5 results = load_results()
6 # График: p1 от V1/V2 при фиксированных затратах
7 filtered = results[(results.c_a .== 1.0) .& (results.c_d .== 1.0), :]
8 ratio = filtered.V1 ./ filtered.V2
9 scatter(
10     ratio,
11     filtered.p_1,
12     group = filtered.type,
13     xlabel = "V1 / V2",
14     ylabel = "p1 (вероятность атаки на актив 1)",
15     title = "Стратегия Нападающего (c_a=1, c_d=1)",
16     legend = :topright,
17 )
18 savefig(plotsdir("p1_vs_ratio.png"))
19
20 # Тепловая карта выигрыша Нападающего
21 grp = groupby(filtered, [:V1, :V2])
22 summ = combine(grp, :UA => mean => :UA_mean)
23 heatmap(
24     sort(unique(summ.V1)),
25     sort(unique(summ.V2)),
26     (x, y) -> summ[(summ.V1 .== x) .& (summ.V2 .== y), :UA_mean][1],
27     xlabel = "V1",
28     ylabel = "V2",
29     title = "Средний выигрыш Нападающего",
30 )
31 savefig(plotsdir("heatmap_UA.png"))
```

Рисунок 3.5: Скрипт построения графиков

```
cd(joinpath(@_DIR_, "../project"))
using DrWatson
@quickactivate "project"
using Plots, DataFrames, Statistics
include(scrdir("simulation.jl"))
results = load_results()
```

[illegible]

График: p1 от V1/V2 при фиксированных затратах

```
filtered = results[(results.c_a == 1.0) & (results.c_d == 1.0), :]  
ratio = filtered.V1 ./ filtered.V2  
scatter(  
    ratio,  
    filtered.p_1,  
    group = filtered.type,  
    xlabel = "V1 / V2",  
    ylabel = "p1 (вероятность атаки на актив 1)",  
    title = "Стратегия Нападающего (c_a=1, c_d=1)",  
    legend = :topright,  
)  
savefig(plotsdir("p1_vs_ratio.png"))
```

"C:\\Users\\arist\\Github\\2026-1--study--security-modeling\\labs\\lab04\\project\\p1"

Тепловая карта выигрыша Нападающего

```
grp = groupby(filtered, [:V1, :V2])  
summ = combine(grp, :UA => mean => :UA_mean)  
heatmap(  
    sort(unique(summ.V1)),  
    sort(unique(summ.V2)),  
    (x, y) -> summ[(summ.V1 == x) & (summ.V2 == y), :UA_mean][1],  
    xlabel = "V1",  
    ylabel = "V2",  
    title = "Средний выигрыш Нападающего",  
)  
savefig(plotsdir("heatmap_UA.png"))
```


"C:\\Users\\arist\\Github\\2026-1--study--security-modeling\\labs\\lab04\\project\\pl

Запускаем этот скрипт:

```
> julia .\src\plot_results.jl
Activating project at 'C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab04\project'
PS C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab04\project
>
```

Рисунок 3.6: Запуск скрипта

В результате выполнения *scripts/plot_results.jl* получаем следующие графики:

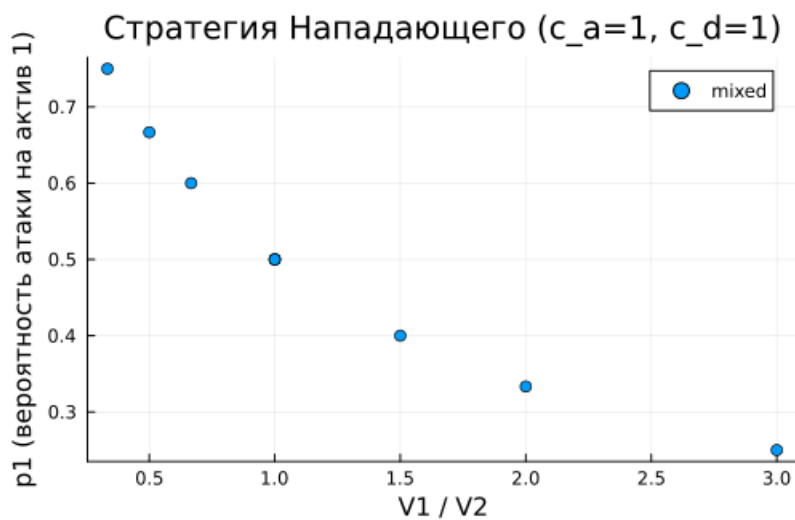


Рисунок 3.7: Зависимость вероятности атаки на первый актив p_1 от отношения ценностей V_1/V_2

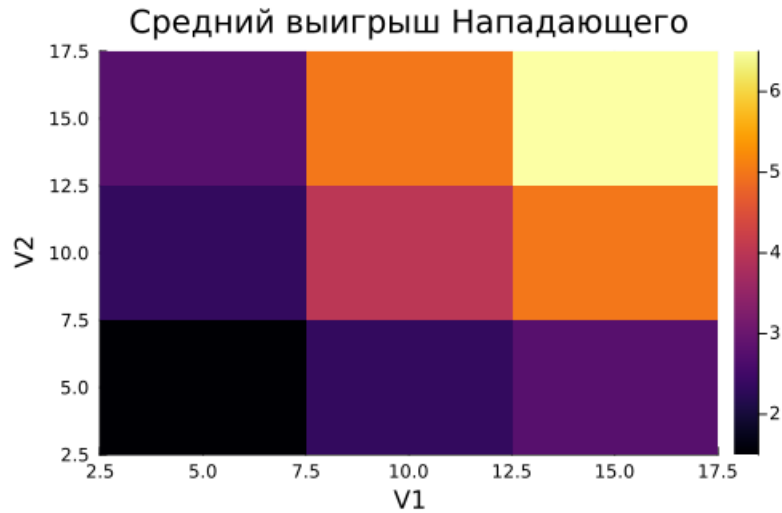


Рисунок 3.8: Средний выигрыш Нападающего

Генерируем производные форматы с помощью скрипта *tangle.jl*:

```
PS C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab04\project
> julia .\src\tangle.jl .\src\simulation.jl
Activating project at 'C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab04\project'
Генерация из: .\src\simulation.jl
[ Info: generating plain script file from 'C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab04\project\src\simulation.jl'
[ Info: writing result to 'C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab04\project\scripts\simulation\simulation.jl'
✓ Чистый скрипт: C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab04\project\scripts\simulation\simulation.jl
[ Info: generating markdown page from 'C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab04\project\src\simulation.jl'
[ Info: writing result to 'C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab04\project\markdown\simulation\simulation.qmd'
```

Рисунок 3.9: Запуск генерации производных форматов

Получаем файлы других форматов: Jupyter notebook, .qmd - Quarto markdown:

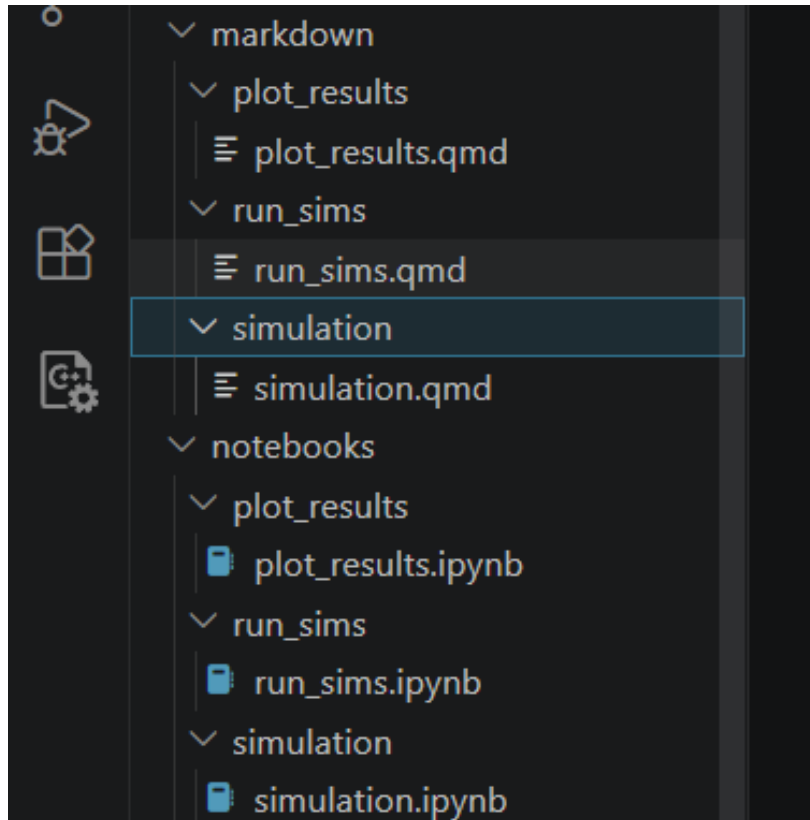


Рисунок 3.10: Полученные файлы других форматов

3.5 Контрольные вопросы

1. Что такое антагонистическая игра? Почему модель «Защитник–Нападающий» можно считать игрой с нулевой суммой?

Антагонистическая игра - игра с прямо противоположными интересами игроков: выигрыш одного равен проигрышу другого.

Модель является игрой с **нулевой суммой**, так как сумма выигрышей постоянна: - При успешной атаке: $(V_i - c_a) + (-V_i - c_d) = -c_a - c_d$ - При отражённой атаке: $(-c_a) + (-c_d) = -c_a - c_d$

Выигрыш Нападающего - это ущерб Защитника, что отражает антагонизм их интересов.

2. Дайте определение равновесия Нэша в чистых и смешанных стратегиях. Приведите пример, когда равновесия в чистых стратегиях не существует.

Равновесие Нэша - набор стратегий, при котором ни одному игроку не выгодно отклоняться в одностороннем порядке.

- **В чистых стратегиях:** каждый игрок выбирает одно действие с вероятностью 1
- **В смешанных стратегиях:** игроки рандомизируют действия, выбирая вероятностное распределение

Пример отсутствия чистого равновесия - игра «Орлянка»: Орел Решка
Орел 1 -1 Решка -1 1 Ни одна из четырёх клеток не является равновесием, но существует смешанное равновесие (0.5, 0.5).

3. Как изменится равновесная стратегия защитника, если ущерб от успешной атаки на незащищённый сервер станет бесконечно большим?

При $V_1 \rightarrow \infty$: - Вероятность защиты первого актива $q_1^* \rightarrow 1$ - Защитник **всегда защищает** критически важный актив - Нападающий вынужден атаковать именно этот актив ($p_1^* \rightarrow 1$)

Практический смысл: критически важные активы (банковские БД, системы управления АЭС) должны защищаться всегда, независимо от затрат.

4. Зачем в информационной безопасности применяют смешанные стратегии? Приведите реальные примеры рандомизации в защите. **Причины применения:**

5. **Непредсказуемость** - противник не может изучить паттерны
6. **Минимизация максимального ущерба** - гарантированный уровень безопасности
7. **Оптимальное распределение ресурсов** - «размазывание» риска

Реальные примеры:

- **Honeypot** - случайное развёртывание ложных сервисов-ловушек
- **ASLR** - случайное размещение кода в памяти ОС
- **Рандомизированные аудиты** - внезапные проверки безопасности
- **Ротация портов** - динамическое перераспределение сетевых сервисов
- **Случайное сканирование** - непредсказуемое время запуска сканеров уязвимостей

4 Выводы

В результате данной лабораторной работы освоено применение теории игр для анализа противостояния в сфере информационной безопасности. Построена матричная игра, найдено равновесие Нэша в чистых и смешанных стратегиях.

Список литературы

1. Описание лабораторных работ